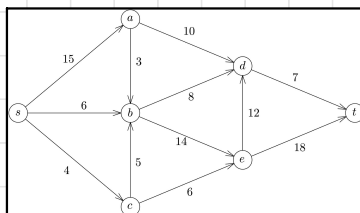


Assignment #2 - Matthew Athanasopoulos

Q1: a)
i)



$$E = \{sa, sb, sc, ab, ad, bd, be, cb, ce, de, ef\}$$

Let x_e be the flow on arc e , $\forall e \in E$

Let u_e be the capacity on arc e , $\forall e \in E$

$$\text{MAX } x_{sa} + x_{sb} + x_{sc}$$

$$\text{s.t. } x_{ab} + x_{ad} - x_{sa} = 0$$

$$x_{bd} + x_{be} - x_{sb} - x_{ab} - x_{cb} = 0$$

$$x_{ce} + x_{cb} - x_{sc} = 0$$

$$x_{de} - x_{ad} - x_{bd} - x_{ed} = 0$$

$$x_{ef} + x_{ed} - x_{be} - x_{ce} = 0$$

$$0 \leq x_e \leq u_e \quad \forall e \in E$$



b)
i)

```
import gurobipy as gp
from gurobipy import GRB

[10]: nodes = ['s','a','b','c','d','e','f']
special = ['s','t']

arcs, cap = gp.multidict({
    ('s','a'): 15,
    ('s','b'): 6,
    ('s','c'): 4,
    ('a','b'): 3,
    ('a','d'): 10,
    ('b','c'): 8,
    ('b','e'): 14,
    ('c','b'): 5,
    ('c','e'): 6,
    ('d','e'): 12,
    ('d','f'): 7,
    ('e','f'): 18,
})

m = gp.Model('MaxFlow')
flow = m.addVars(arcs, name='flow')
m.setObjective(flow.sum('s', '*'), GRB.MAXIMIZE)
m.addConstrs((flow[a] == cap[a] for a in arcs), name='cap')
m.addConstrs((flow.sum('s', '*') - flow.sum('t', '*') == 0 for s in nodes if s not in special), name='conservative')

('a': <gurobi.Constr <Awaiting Model Update>, ...

[20]: m.update()
m.write('cost0_max_flow.lp')
m.optimize()

Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm])

[21]: m.printAttr('x')

Variable      x
flow[s,a]    10
flow[s,b]     6
flow[s,c]     4
flow[a,b]     3
flow[a,d]     9
flow[b,e]     7
flow[c,e]     4
flow[d,e]     4
flow[e,f]    13
```

Solved in 2 iterations and 0.01 seconds (0.00 work units)
Optimal objective 2.000000000e+01

ii)

```
import gurobipy as gp
from gurobipy import GRB

b = {'a': 12,
      'b': 6,
      'c': 8,
      'd': -6,
      'e': -1,
      'f': -11}

arcs = [
    ('a','b'): (10, 5),
    ('a','c'): (10, 2),
    ('a','d'): (100, 9),
    ('b','c'): (100, 9),
    ('b','e'): (80, 8),
    ('c','b'): (60, 7),
    ('c','e'): (80, 8),
    ('d','c'): (60, 7),
    ('d','e'): (20, 15),
    ('e','d'): (-60, 4),
    ('e','f'): (30, 10),
    ('f','e'): (30, 10),
    ('f','d'): (30, 10),
]

m = gp.Model('MinCostFlow')
x = m.addVars(arcs, name='flow', lb=0)
m.addConstrs((x[i] == arcs[i][1] for i in arcs), name='cap')
m.setObjective(gp.quicksum(arcs[i][0] * x[i] for i in arcs), GRB.MINIMIZE)
m.addConstrs(
    gp.quicksum(x[i] for j in b if (v, j) in arcs)
    == b[v] for v in b if (v, j) in arcs
),
    name='balance',
)
m.update()
m.write('cost2_min_flow.lp')
m.optimize()

Warning: Linear constraint 0 and Linear constraint 1 have the same name 'cap[1]'...

m.printAttr('x')

Variable      x
flow[a,b]     3
flow[a,c]     2
flow[a,d]     7
flow[b,c]     5
flow[b,e]     4
flow[c,b]     4
flow[c,e]     4
flow[d,e]     8
flow[f,d]     8
```

Solved in 0 iterations and 0.01 seconds (0.00 work units)
Optimal objective 1.570000000e+01

```
arc_data = {
    ('a','b'): (10, 5),
    ('a','c'): (10, 2),
    ('a','d'): (100, 9),
    ('b','c'): (100, 9),
    ('b','e'): (80, 8),
    ('c','b'): (60, 7),
    ('c','e'): (80, 8),
    ('d','c'): (60, 7),
    ('d','e'): (20, 15),
    ('e','d'): (-60, 4),
    ('e','f'): (30, 10),
    ('f','e'): (30, 10),
    ('f','d'): (30, 10),
}

arc_data['b','c'] = (arc_data['b','c'][0], 5.5) # increase cap (see my notes)
arc_data['c','e'] = (arc_data['c','e'][0], 6.5) # decrease cap (see my notes)

arcs = list(arc_data.keys())
cost = (a: arc_data[a][0] for a in arcs)
cap = (a: arc_data[a][1] for a in arcs)

m = gp.Model('MinCostFlow')
x = m.addVars(arcs, name='flow', lb=0, ub=cap)
m.setObjective(gp.quicksum(cost[a] * x[a] for a in arcs), GRB.MINIMIZE)

v = list(b.keys())
for v in v:
    outflow = gp.quicksum(x[i] for (i,j) in arcs if i == v)
    inflow = gp.quicksum(x[i] for (i,j) in arcs if j == v)
    m.addConstr(outflow - inflow == b[v], name='bal[{}]'(v))

m.update()
m.write('cost2_min_flow.lp')
m.optimize()
```

CHANGED!

c) I decided to use gurobi to find optimal solutions with some fractional flow values with Gurobi. This was done by adjusting the bounds for capacity on various arcs for maxflow and mincost.

i) Maxflow: I first derived an upper bound: All flow reaching t must go through either d-t (cap 7) or e-t. But the most that can reach e from the source side is limited by the available feeders s-b (6), a-b (3), and s-c (4), totalling 13.

Thus max flow = 7 + 13 = 20. I then constructed a feasible flow that attains this bound: Send 7 along s-a-b-t; send 9 to e via s-b and a-b (b-e=9), and the remaining 4 via c-e, then (e-t)=13. To produce a fractional optimal solution with the same value, divert 0.2 from c to b ((c-b)=0.2, (c-e)=3.8) and increase b to 9.2. This keeps the total to t at 20 while introducing fractional arc values!

```
nodes = ['s','a','b','c','d','e','f']
special = ['s','t']

arcs, cap = gp.multidict({
    ('s','a'): 15,
    ('s','b'): 6,
    ('s','c'): 4,
    ('a','b'): 3,
    ('a','d'): 9,
    ('b','c'): 8,
    ('b','e'): 9.2,
    ('c','b'): 0.2,
    ('c','e'): 3.8,
    ('d','e'): 7,
    ('d','f'): 12,
    ('e','d'): -1,
    ('e','f'): 18,
})

m = gp.Model('MaxFlow')
flow = m.addVars(arcs, name='flow')
m.setObjective(flow.sum('s', '*'), GRB.MAXIMIZE)
m.addConstrs((flow[a] == cap[a] for a in arcs), name='cap')
m.addConstrs((flow.sum('s', '*') - flow.sum('t', '*') == 0 for s in nodes if s not in special), name='conservative')
```

Solved in 0 iterations and 0.01 seconds (0.00 work units)
Optimal objective 2.000000000e+01

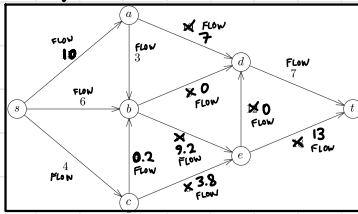
ii) Solved the base min-cost flow to optimality (subroute d-c, use cheapest feeders to e, split d via equal cost routes). To exhibit a fractional optimum, I nudged bounds slightly: raised b-c and lowered a-b by the same amount - shifting 0.5 units onto the multi-arc path. cost stays the same; all balances/caps hold, so it's a valid fractional optimal solution. And see gurobi adjustment above 7.

Variable	x
flow[a,b]	3.5
flow[a,c]	2
flow[a,d]	6.5
flow[b,c]	5.5
flow[b,e]	4
flow[c,b]	9.5
flow[c,e]	4
flow[d,e]	8
flow[f,d]	8.5

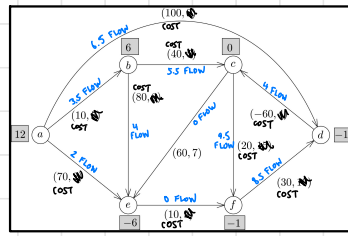
Solved in 0 iterations and 0.01 seconds (0.00 work units)
Optimal objective 1.570000000e+01

Q1c) Drawings:

i)



ii)



Q2:

There are n teams $T = \{1, \dots, n\}$ in the company, where team i has a_i employees.

There are two group chat sessions, where employees are divided into m groups $G = \{1, \dots, m\}$

↳ Group j has at most b_j employees.

In the first session, each team must send all their employees into different groups.

In the second session, everyone must move to a different group, and at MOST 2 people in the same group from the first session can go to each group for the second session.

My Solution:

Notes:

Source: S

Teams: $T_1, \dots, T_n$

Session-1 groups: $G_1^{(1)}, \dots, G_m^{(1)}$

Session-1 capacity gates: $C_1, \dots, C_m$

Session-2 groups: $G_1^{(2)}, \dots, G_m^{(2)}$

Sink: t

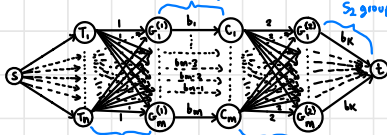
Arcs and capacities:

- $S \rightarrow T_i$ with cap a_i for all i .
- $T_i \rightarrow G_j^{(1)}$ with cap 1 for all i, j .
- $G_j^{(1)} \rightarrow C_j$ with cap b_j for all j (bottleneck for S1 size).
- $C_j \rightarrow G_k^{(2)}$ with cap 2 for all $j \neq k$. (omit $j=k$ to force "different group" across sessions)
- $G_k^{(2)} \rightarrow t$ with cap b_k for all k (S2 group capacities).

Sketch:

Cap b_j : total assigned to S2 group j is $\leq b_j$.
The bottleneck enforces the group size limit: caps total people assigned to S_2 group j (via $G_j^{(1)} \rightarrow C_j$)!

Cap b_k : S_2 group k holds $\leq b_k$ in total.
The bottleneck enforces the group size limit: caps total people assigned to S_2 group k (via $G_k^{(2)} \rightarrow t$)!



For all arcs between T_i and $G_j^{(1)}$, there exists capacity 1, ensuring all members of team i go to different groups.

For all arcs between C_j and $G_k^{(2)}$, there exists capacity 2, ensuring at most 2 people from the first session can go to each group for the second session.

Additional Explanation:

• Units: people; integrality: all capacities are integers, so any max flow yields integer flows (whole people).

How optimal sol'n answers the question:

• Let $A = \sum_i a_i$ be the total number of employees.

↳ If the maximum s-t flow = A , then a valid two-session assignment exists; read each $S \rightarrow T_i \rightarrow G_j^{(1)} \rightarrow C_j \rightarrow G_k^{(2)} \rightarrow t$ path as one person's schedule (S_1 in j , S_2 in k).

↳ If the max flow $< A$, such an assignment is impossible. The corresponding minimum cut pinpoints the blocking constraint(s): too small b_j / b_k , too few groups relative to some a_i , or the pairwise " ≤ 2 " limits.

Sanity Conditions:

Just may be helpful to note...

↳ must have $m \geq \max_i a_i$ (else a team can't split across S_1 groups)

↳ Also $\sum_j b_j \geq A$ and $\sum_k b_k \geq A$.

Q3:

Let n be the number of months; months are $i = 1, \dots, n$.

Let d_i represent the demand (headcount) for each month $i \in \{1, 2, 3, 4, 5, 6\}$

Let S_3 and S_5 represent the cost of hiring one person for 3 and 5 consecutive months respectively.

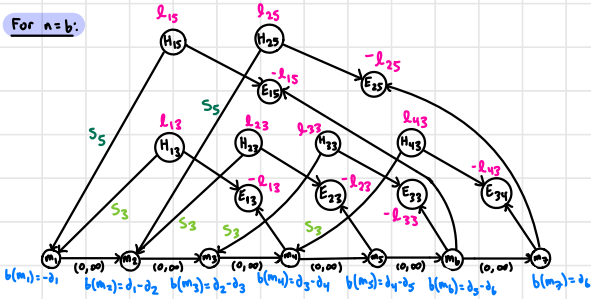
Let l_{ij} be the available workers in month i coming from the j month cohort.

Let H_{ij} be the hires made in month i on a j -month contract. (3 or 5 month groups)

Let E_{ij} be exit/spillover from cohort (i, j) - workers removed when contract end or cohort has surplus.

$j \in \{3, 5\}$

For $n = 6$:



Explanation:

We enforce monthly minimums by adding large negative-cost ($-M$) arcs from each month node to sink. That "reward" makes the solver push at least d_i units through every month node: so staffing floors are met without explicit lower bounds. Hiring cost is paid exactly once on $S \rightarrow H_{ij}$; continuations are free, and the month $\rightarrow$ sink's negatives ensure demand is satisfied.

Contract length comes from chaining E nodes: a unit must serve exactly 3 or 5 consecutive months. No leaving work early or staying longer. As long as total capacity $\geq$ total demand, the network is feasible, and the optimizer picks the cheapest mix of 3 and 5 contracts.